

```
In [ ]: from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=20,      # Повороты до 20 градусов
    shear_range=0.2,       # Сдвиги углов (shear)
    zoom_range=0.2,        # Зум до 20%
    horizontal_flip=True,  # Горизонтальное отражение
    validation_split=0.2,  # 80% для обучения, 20% для валидации
    rescale=1./255,
    width_shift_range=0.2, # Горизонтальные сдвиги
    height_shift_range=0.2, # Вертикальные сдвиги
    fill_mode='nearest'    # Заполнение пикселей при сдвигах
)

# Генератор для тренировочной выборки
train_generator = datagen.flow_from_directory(
    'C:/Users/Andrey/Desktop/flowers', # путь к общей папке
    target_size=(128, 128),           # Преобразуем изображения к размеру 128x128
    batch_size=32,                    # Размер пакета
    class_mode='categorical',         # Классификация на несколько классов
    subset='training'                 # Используем 80% данных
)

# Генератор для валидационной выборки
val_generator = datagen.flow_from_directory(
    'C:/Users/Andrey/Desktop/flowers',
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical',
    subset='validation'               # Используем 20% данных
)
```

Found 3457 images belonging to 5 classes.
Found 860 images belonging to 5 classes.

```
In [7]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model_cnn = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax') # 5 классов
])
model_cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

C:\anaconda\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
In [47]: history = model_cnn.fit(train_generator, epochs=20, validation_data=val_generator)
```

```

Epoch 1/20
109/109 ————— 38s 335ms/step - accuracy: 0.3471 - loss: 1.4556 - val_accuracy: 0.5570 - val_loss
: 1.1457
Epoch 2/20
109/109 ————— 41s 379ms/step - accuracy: 0.5227 - loss: 1.1453 - val_accuracy: 0.6174 - val_loss
: 1.0124
Epoch 3/20
109/109 ————— 39s 357ms/step - accuracy: 0.6025 - loss: 1.0059 - val_accuracy: 0.6395 - val_loss
: 0.9451
Epoch 4/20
109/109 ————— 41s 373ms/step - accuracy: 0.6215 - loss: 0.9489 - val_accuracy: 0.6256 - val_loss
: 0.9418
Epoch 5/20
109/109 ————— 41s 372ms/step - accuracy: 0.5988 - loss: 1.0259 - val_accuracy: 0.6744 - val_loss
: 0.8320
Epoch 6/20
109/109 ————— 37s 341ms/step - accuracy: 0.6500 - loss: 0.9072 - val_accuracy: 0.6523 - val_loss
: 0.8939
Epoch 7/20
109/109 ————— 37s 336ms/step - accuracy: 0.6639 - loss: 0.8716 - val_accuracy: 0.6756 - val_loss
: 0.8536
Epoch 8/20
109/109 ————— 36s 330ms/step - accuracy: 0.6543 - loss: 0.8969 - val_accuracy: 0.6895 - val_loss
: 0.8241
Epoch 9/20
109/109 ————— 38s 352ms/step - accuracy: 0.6912 - loss: 0.8352 - val_accuracy: 0.6767 - val_loss
: 0.8143
Epoch 10/20
109/109 ————— 37s 342ms/step - accuracy: 0.6869 - loss: 0.8282 - val_accuracy: 0.7035 - val_loss
: 0.7820
Epoch 11/20
109/109 ————— 37s 341ms/step - accuracy: 0.7093 - loss: 0.7742 - val_accuracy: 0.7012 - val_loss
: 0.7607
Epoch 12/20
109/109 ————— 36s 334ms/step - accuracy: 0.7180 - loss: 0.7513 - val_accuracy: 0.7140 - val_loss
: 0.7741
Epoch 13/20
109/109 ————— 37s 339ms/step - accuracy: 0.7220 - loss: 0.7447 - val_accuracy: 0.7163 - val_loss
: 0.7659
Epoch 14/20
109/109 ————— 38s 352ms/step - accuracy: 0.7330 - loss: 0.7205 - val_accuracy: 0.7012 - val_loss
: 0.7510
Epoch 15/20
109/109 ————— 36s 332ms/step - accuracy: 0.7317 - loss: 0.7189 - val_accuracy: 0.7140 - val_loss
: 0.7497
Epoch 16/20
109/109 ————— 36s 333ms/step - accuracy: 0.7438 - loss: 0.6905 - val_accuracy: 0.7291 - val_loss
: 0.7196
Epoch 17/20
109/109 ————— 42s 386ms/step - accuracy: 0.7345 - loss: 0.7048 - val_accuracy: 0.7500 - val_loss
: 0.7535
Epoch 18/20
109/109 ————— 40s 365ms/step - accuracy: 0.7521 - loss: 0.6473 - val_accuracy: 0.7209 - val_loss
: 0.7625
Epoch 19/20
109/109 ————— 36s 326ms/step - accuracy: 0.7379 - loss: 0.7061 - val_accuracy: 0.7640 - val_loss
: 0.6768
Epoch 20/20
109/109 ————— 39s 361ms/step - accuracy: 0.7748 - loss: 0.6563 - val_accuracy: 0.7256 - val_loss
: 0.7573

```

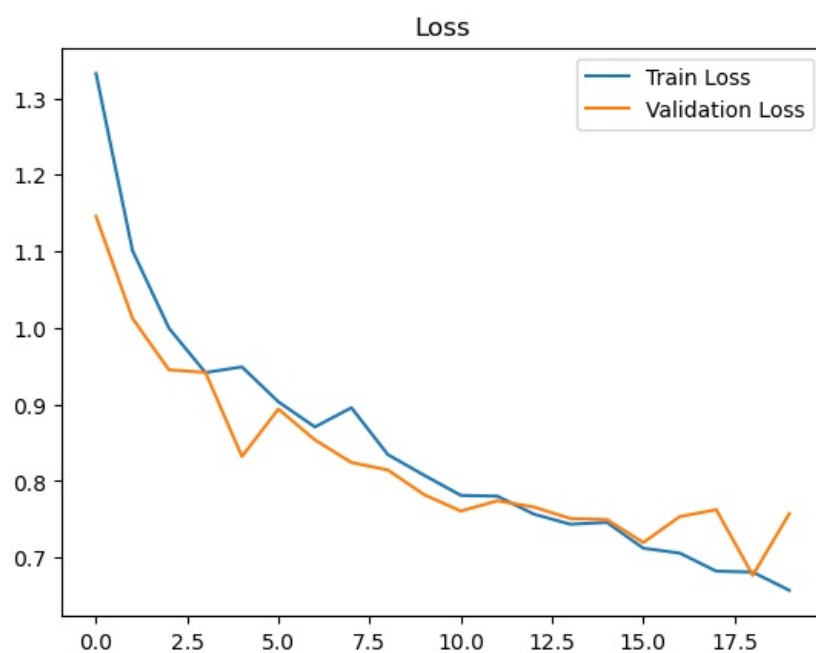
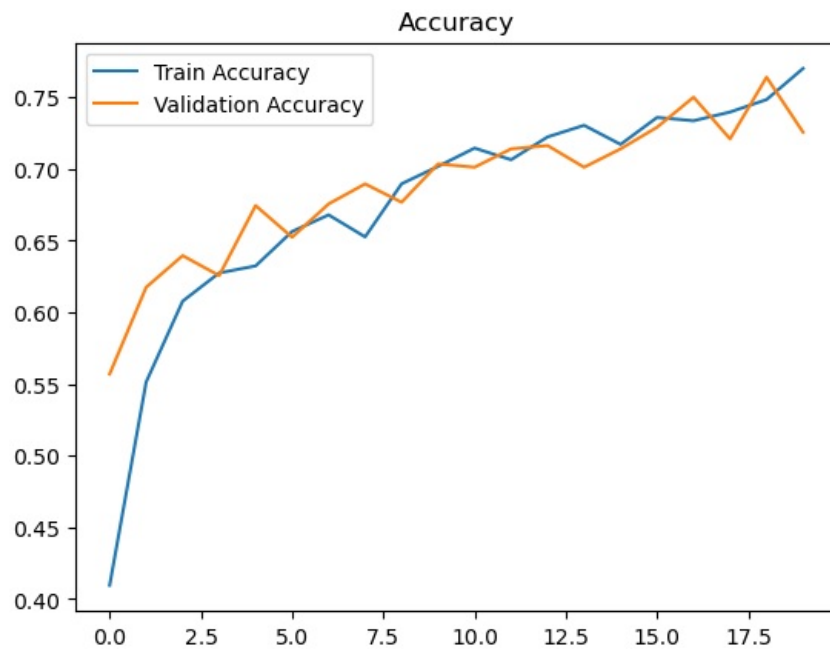
```
In [51]: import matplotlib.pyplot as plt
```

```

# График точности
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
plt.show()

# График потерь
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.show()

```



```
In [ ]: from tensorflow.keras.optimizers import Adam

model_fcn = Sequential([
    # Входной слой - изображение преобразуется в одномерный вектор
    Flatten(input_shape=(128, 128, 3)), # Преобразуем изображение 128x128x3 в одномерный вектор
    # Первый полносвязный слой
    Dense(512, activation='relu'), # 512 нейронов
    Dropout(0.2), # Регуляризация через dropout

    # Второй полносвязный слой
    Dense(256, activation='relu'), # 256 нейронов
    Dropout(0.2), # Регуляризация через dropout

    Dense(256, activation='relu'),
    Dropout(0.2), # Регуляризация через dropout
    Dense(128, activation='relu'),
    Dropout(0.2), # Регуляризация через dropout
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    # Выходной слой для классификации на 5 классов
    Dense(5, activation='softmax') # 5 классов, softmax для вероятностей
])

# Компиляция модели
model_fcn.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
```

C:\anaconda\lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
 super().__init__(**kwargs)

```
In [78]: history_fcn = model_fcn.fit(train_generator, epochs=20, validation_data=val_generator)
```

```

Epoch 1/20
109/109 ————— 53s 472ms/step - accuracy: 0.2420 - loss: 3.9965 - val_accuracy: 0.3326 - val_loss
: 1.5107
Epoch 2/20
109/109 ————— 60s 550ms/step - accuracy: 0.3538 - loss: 1.4402 - val_accuracy: 0.4256 - val_loss
: 1.3682
Epoch 3/20
109/109 ————— 52s 473ms/step - accuracy: 0.3903 - loss: 1.3893 - val_accuracy: 0.3698 - val_loss
: 1.4004
Epoch 4/20
109/109 ————— 45s 414ms/step - accuracy: 0.3995 - loss: 1.3658 - val_accuracy: 0.3919 - val_loss
: 1.3947
Epoch 5/20
109/109 ————— 42s 390ms/step - accuracy: 0.3955 - loss: 1.3706 - val_accuracy: 0.3942 - val_loss
: 1.3488
Epoch 6/20
109/109 ————— 46s 423ms/step - accuracy: 0.3936 - loss: 1.3372 - val_accuracy: 0.3942 - val_loss
: 1.3459
Epoch 7/20
109/109 ————— 41s 376ms/step - accuracy: 0.4302 - loss: 1.2997 - val_accuracy: 0.3802 - val_loss
: 1.3764
Epoch 8/20
109/109 ————— 37s 338ms/step - accuracy: 0.3947 - loss: 1.3456 - val_accuracy: 0.4174 - val_loss
: 1.2988
Epoch 9/20
109/109 ————— 40s 371ms/step - accuracy: 0.4210 - loss: 1.3112 - val_accuracy: 0.4163 - val_loss
: 1.3246
Epoch 10/20
109/109 ————— 44s 404ms/step - accuracy: 0.3922 - loss: 1.3099 - val_accuracy: 0.3860 - val_loss
: 1.3582
Epoch 11/20
109/109 ————— 54s 491ms/step - accuracy: 0.4141 - loss: 1.3238 - val_accuracy: 0.4093 - val_loss
: 1.3344
Epoch 12/20
109/109 ————— 54s 493ms/step - accuracy: 0.4248 - loss: 1.2916 - val_accuracy: 0.4302 - val_loss
: 1.2752
Epoch 13/20
109/109 ————— 48s 442ms/step - accuracy: 0.4451 - loss: 1.2708 - val_accuracy: 0.4221 - val_loss
: 1.2953
Epoch 14/20
109/109 ————— 47s 426ms/step - accuracy: 0.4117 - loss: 1.2997 - val_accuracy: 0.4523 - val_loss
: 1.2814
Epoch 15/20
109/109 ————— 44s 406ms/step - accuracy: 0.4310 - loss: 1.2589 - val_accuracy: 0.4337 - val_loss
: 1.2875
Epoch 16/20
109/109 ————— 46s 419ms/step - accuracy: 0.4345 - loss: 1.2759 - val_accuracy: 0.4302 - val_loss
: 1.2994
Epoch 17/20
109/109 ————— 53s 485ms/step - accuracy: 0.4392 - loss: 1.2753 - val_accuracy: 0.4337 - val_loss
: 1.3002
Epoch 18/20
109/109 ————— 59s 540ms/step - accuracy: 0.4468 - loss: 1.2498 - val_accuracy: 0.3860 - val_loss
: 1.3886
Epoch 19/20
109/109 ————— 43s 392ms/step - accuracy: 0.4285 - loss: 1.2722 - val_accuracy: 0.4326 - val_loss
: 1.2706
Epoch 20/20
109/109 ————— 38s 349ms/step - accuracy: 0.4495 - loss: 1.2496 - val_accuracy: 0.4221 - val_loss
: 1.3234

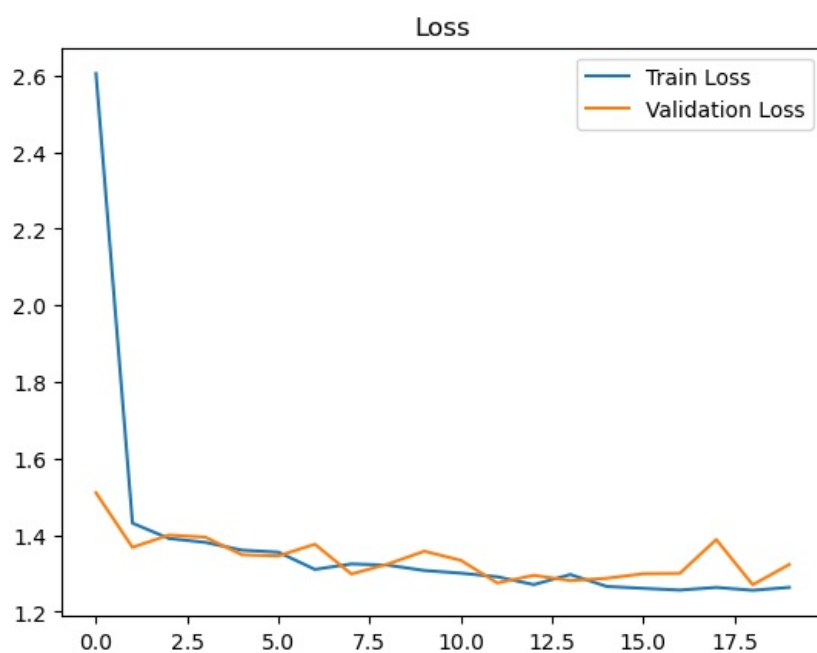
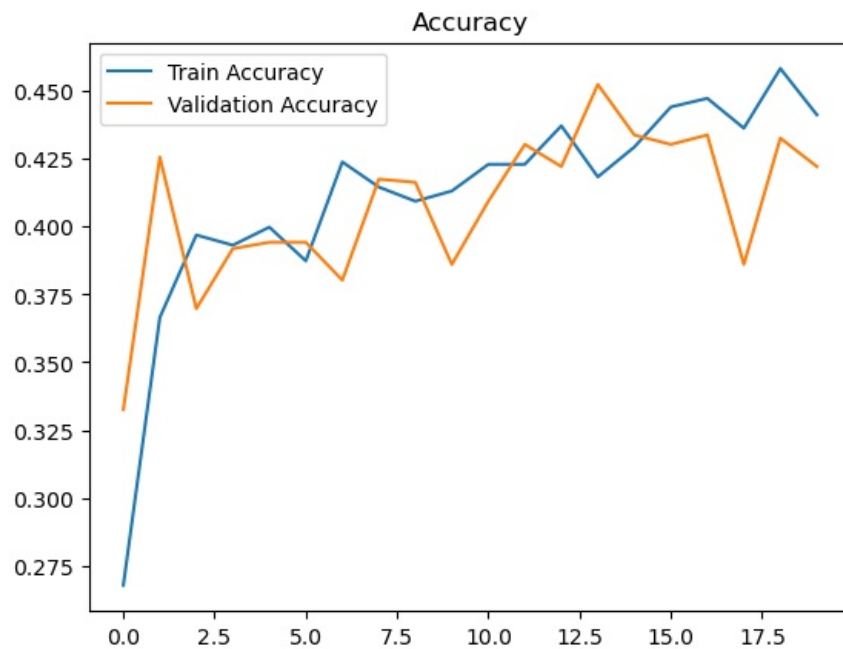
```

```

In [82]: # График точности
plt.plot(history_fcn.history['accuracy'], label='Train Accuracy')
plt.plot(history_fcn.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
plt.show()

# График потерь
plt.plot(history_fcn.history['loss'], label='Train Loss')
plt.plot(history_fcn.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.show()

```



```
In [ ]: autoencoder_datagen = ImageDataGenerator(
    rotation_range=20,      # Повороты до 20 градусов
    shear_range=0.2,       # Сдвиги углов (shear)
    zoom_range=0.2,        # Зум до 20%
    horizontal_flip=True,  # Горизонтальное отражение
    validation_split=0.2,   # 80% для обучения, 20% для валидации
    rescale=1./255,
    width_shift_range=0.2,  # Горизонтальные сдвиги
    height_shift_range=0.2, # Вертикальные сдвиги
    fill_mode='nearest'    # Заполнение пикселей при сдвигах
)

# Генератор для тренировочной выборки
autoencoder_train_generator = datagen.flow_from_directory(
    'C:/Users/Andrey/Desktop/flowers',      # путь к общей папке
    target_size=(128, 128),                # Преобразуем изображения к размеру 128x128
    batch_size=32,                          # Размер пакета
    class_mode='input',                    # Классификация на несколько классов
    subset='training'                      # Используем 80% данных
)

# Генератор для валидационной выборки
autoencoder_val_generator = datagen.flow_from_directory(
    'C:/Users/Andrey/Desktop/flowers',
    target_size=(128, 128),
    batch_size=32,
    class_mode='input',
    subset='validation'                    # Используем 20% данных
)
```

Found 3457 images belonging to 5 classes.
Found 860 images belonging to 5 classes.

```
In [11]: from tensorflow.keras.layers import Reshape
```

```

from tensorflow.keras.layers import Reshape
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Энкодер (сжимающий слой)
autoencoder_encoder = Sequential([
    Flatten(input_shape=(128, 128, 3)), # Преобразуем изображение в одномерный вектор
    Dense(256, activation='relu') # Сжимаем изображение до 256 признаков
])

# Декодер (восстановление)
autoencoder_decoder = Sequential([
    Dense(128 * 128 * 3, activation='sigmoid', input_shape=(256,)), # Восстановление изображения
    Reshape((128, 128, 3)) # Возвращаем форму исходного изображения
])

# Полный автокодировщик (энкодер + декодер)
autoencoder = Sequential([autoencoder_encoder, autoencoder_decoder])

# Компиляция модели
autoencoder.compile(loss="mse", optimizer='adam')

# Обучение автокодировщика
autoencoder.fit(
    autoencoder_train_generator, # Генератор обучающих данных
    epochs=40,
    validation_data=autoencoder_val_generator
)

```

C:\anaconda\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(**kwargs)
C:\anaconda\Lib\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
C:\anaconda\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.
self.warn_if_super_not_called()

Epoch 1/40
109/109  0s 385ms/step - loss: 0.0741

C:\anaconda\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.
self.warn_if_super_not_called()

```

109/109 ————— 51s 456ms/step - loss: 0.0740 - val_loss: 0.0573
Epoch 2/40
109/109 ————— 35s 316ms/step - loss: 0.0540 - val_loss: 0.0537
Epoch 3/40
109/109 ————— 34s 312ms/step - loss: 0.0503 - val_loss: 0.0503
Epoch 4/40
109/109 ————— 37s 341ms/step - loss: 0.0479 - val_loss: 0.0495
Epoch 5/40
109/109 ————— 39s 355ms/step - loss: 0.0481 - val_loss: 0.0491
Epoch 6/40
109/109 ————— 35s 318ms/step - loss: 0.0475 - val_loss: 0.0475
Epoch 7/40
109/109 ————— 38s 351ms/step - loss: 0.0469 - val_loss: 0.0466
Epoch 8/40
109/109 ————— 38s 347ms/step - loss: 0.0453 - val_loss: 0.0450
Epoch 9/40
109/109 ————— 38s 351ms/step - loss: 0.0445 - val_loss: 0.0488
Epoch 10/40
109/109 ————— 34s 315ms/step - loss: 0.0458 - val_loss: 0.0463
Epoch 11/40
109/109 ————— 39s 355ms/step - loss: 0.0530 - val_loss: 0.0467
Epoch 12/40
109/109 ————— 38s 353ms/step - loss: 0.0450 - val_loss: 0.0489
Epoch 13/40
109/109 ————— 38s 352ms/step - loss: 0.0465 - val_loss: 0.0454
Epoch 14/40
109/109 ————— 39s 354ms/step - loss: 0.0457 - val_loss: 0.0457
Epoch 15/40
109/109 ————— 39s 357ms/step - loss: 0.0435 - val_loss: 0.0480
Epoch 16/40
109/109 ————— 39s 359ms/step - loss: 0.0449 - val_loss: 0.0468
Epoch 17/40
109/109 ————— 39s 359ms/step - loss: 0.0431 - val_loss: 0.0440
Epoch 18/40
109/109 ————— 40s 365ms/step - loss: 0.0427 - val_loss: 0.0426
Epoch 19/40
109/109 ————— 37s 339ms/step - loss: 0.0423 - val_loss: 0.0441
Epoch 20/40
109/109 ————— 36s 328ms/step - loss: 0.0420 - val_loss: 0.0425
Epoch 21/40
109/109 ————— 37s 338ms/step - loss: 0.0411 - val_loss: 0.0431
Epoch 22/40
109/109 ————— 37s 337ms/step - loss: 0.0414 - val_loss: 0.0423
Epoch 23/40
109/109 ————— 37s 341ms/step - loss: 0.0423 - val_loss: 0.0417
Epoch 24/40
109/109 ————— 36s 332ms/step - loss: 0.0402 - val_loss: 0.0433
Epoch 25/40
109/109 ————— 37s 335ms/step - loss: 0.0414 - val_loss: 0.0419
Epoch 26/40
109/109 ————— 36s 330ms/step - loss: 0.0414 - val_loss: 0.0416
Epoch 27/40
109/109 ————— 37s 337ms/step - loss: 0.0408 - val_loss: 0.0405
Epoch 28/40
109/109 ————— 35s 323ms/step - loss: 0.0400 - val_loss: 0.0400
Epoch 29/40
109/109 ————— 36s 331ms/step - loss: 0.0390 - val_loss: 0.0409
Epoch 30/40
109/109 ————— 37s 341ms/step - loss: 0.0401 - val_loss: 0.0394
Epoch 31/40
109/109 ————— 36s 331ms/step - loss: 0.0390 - val_loss: 0.0430
Epoch 32/40
109/109 ————— 36s 329ms/step - loss: 0.0393 - val_loss: 0.0391
Epoch 33/40
109/109 ————— 36s 329ms/step - loss: 0.0390 - val_loss: 0.0405
Epoch 34/40
109/109 ————— 36s 330ms/step - loss: 0.0393 - val_loss: 0.0389
Epoch 35/40
109/109 ————— 36s 328ms/step - loss: 0.0383 - val_loss: 0.0380
Epoch 36/40
109/109 ————— 36s 330ms/step - loss: 0.0395 - val_loss: 0.0392
Epoch 37/40
109/109 ————— 36s 331ms/step - loss: 0.0384 - val_loss: 0.0384
Epoch 38/40
109/109 ————— 36s 331ms/step - loss: 0.0382 - val_loss: 0.0388
Epoch 39/40
109/109 ————— 38s 346ms/step - loss: 0.0379 - val_loss: 0.0380
Epoch 40/40
109/109 ————— 36s 327ms/step - loss: 0.0374 - val_loss: 0.0381

```

```
Out[11]: <keras.src.callbacks.history.History at 0x25d73d24410>
```

```
In [13]: from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
```

```
# Извлечение энкодера из автокодировщика
autoencoder = autoencoder_encoder
```

```
# Замораживание весов энкодера, чтобы не переобучать уже обученные слои
autoencoder.trainable = False
```

```
# Создание классификатора
```

```
classifier = Sequential([
    autoencoder, # Используем энкодер как первый слой
    Dense(256, activation="relu"), # Добавляем полносвязный слой
    Dense(256, activation="relu"), # Добавляем полносвязный слой
    Dense(128, activation="relu"), # Добавляем полносвязный слой
    Dense(128, activation="relu"), # Добавляем полносвязный слой
    Dense(64, activation="relu"), # Добавляем полносвязный слой
    Dense(32, activation="relu"), # Добавляем полносвязный слой
    #Dropout(0.5), # Dropout для предотвращения переобучения
    Dense(5, activation="softmax") # Выходной слой для 5 классов
])
```

```
# Компиляция модели
```

```
classifier.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
# Обучение классификатора
```

```
history = classifier.fit(
    train_generator,
    epochs=20,
    validation_data=val_generator
)
```

```

Epoch 1/20
109/109 ————— 35s 293ms/step - accuracy: 0.3257 - loss: 1.5222 - val_accuracy: 0.3663 - val_loss
: 1.3782
Epoch 2/20
109/109 ————— 31s 284ms/step - accuracy: 0.3723 - loss: 1.3859 - val_accuracy: 0.4012 - val_loss
: 1.3540
Epoch 3/20
109/109 ————— 31s 283ms/step - accuracy: 0.3884 - loss: 1.3554 - val_accuracy: 0.4198 - val_loss
: 1.3196
Epoch 4/20
109/109 ————— 31s 285ms/step - accuracy: 0.4241 - loss: 1.3156 - val_accuracy: 0.4058 - val_loss
: 1.3226
Epoch 5/20
109/109 ————— 24s 216ms/step - accuracy: 0.4278 - loss: 1.3116 - val_accuracy: 0.3977 - val_loss
: 1.3256
Epoch 6/20
109/109 ————— 19s 170ms/step - accuracy: 0.4232 - loss: 1.2966 - val_accuracy: 0.4198 - val_loss
: 1.3265
Epoch 7/20
109/109 ————— 18s 169ms/step - accuracy: 0.4265 - loss: 1.2823 - val_accuracy: 0.4209 - val_loss
: 1.3039
Epoch 8/20
109/109 ————— 18s 169ms/step - accuracy: 0.4387 - loss: 1.2842 - val_accuracy: 0.4047 - val_loss
: 1.3057
Epoch 9/20
109/109 ————— 18s 169ms/step - accuracy: 0.4358 - loss: 1.2963 - val_accuracy: 0.4279 - val_loss
: 1.2947
Epoch 10/20
109/109 ————— 18s 169ms/step - accuracy: 0.4156 - loss: 1.3182 - val_accuracy: 0.4465 - val_loss
: 1.2989
Epoch 11/20
109/109 ————— 19s 176ms/step - accuracy: 0.4288 - loss: 1.2930 - val_accuracy: 0.4523 - val_loss
: 1.2709
Epoch 12/20
109/109 ————— 19s 174ms/step - accuracy: 0.4658 - loss: 1.2647 - val_accuracy: 0.4349 - val_loss
: 1.2985
Epoch 13/20
109/109 ————— 18s 170ms/step - accuracy: 0.4413 - loss: 1.2699 - val_accuracy: 0.4140 - val_loss
: 1.3062
Epoch 14/20
109/109 ————— 19s 173ms/step - accuracy: 0.4540 - loss: 1.2759 - val_accuracy: 0.4477 - val_loss
: 1.2586
Epoch 15/20
109/109 ————— 18s 169ms/step - accuracy: 0.4595 - loss: 1.2561 - val_accuracy: 0.4244 - val_loss
: 1.2944
Epoch 16/20
109/109 ————— 19s 170ms/step - accuracy: 0.4521 - loss: 1.2685 - val_accuracy: 0.4070 - val_loss
: 1.3014
Epoch 17/20
109/109 ————— 18s 168ms/step - accuracy: 0.4519 - loss: 1.2703 - val_accuracy: 0.4384 - val_loss
: 1.2609
Epoch 18/20
109/109 ————— 19s 171ms/step - accuracy: 0.4388 - loss: 1.2877 - val_accuracy: 0.4512 - val_loss
: 1.2692
Epoch 19/20
109/109 ————— 18s 169ms/step - accuracy: 0.4637 - loss: 1.2393 - val_accuracy: 0.4140 - val_loss
: 1.2963
Epoch 20/20
109/109 ————— 19s 170ms/step - accuracy: 0.4499 - loss: 1.2792 - val_accuracy: 0.4407 - val_loss
: 1.3064

```

```

In [ ]: from tensorflow.keras.preprocessing import image
import numpy as np
# Загрузка и подготовка изображения
img_path = 'C:/Users/Andrey/Downloads/krasivyi-buket-romashek-kartinka-4.webp' # Укажите путь к вашему изображ
img = image.load_img(img_path, target_size=(128, 128)) # Измените размер под вход модели
img_array = image.img_to_array(img) # Преобразуйте изображение в массив
img_array = np.expand_dims(img_array, axis=0) # Добавьте размерность для батча

# Нормализация значений пикселей
img_array /= 255.0 # Нормализация

# Прогнозирование
predictions = classifier.predict(img_array) # Передача изображения в predict
predicted_class = np.argmax(predictions, axis=1) # Получение индекса класса с максимальной вероятностью

# Интерпретация результатов
class_names = ['daisy', 'dandelion', 'rose', 'sunflower', 'tulip'] # Замените на ваши имена классов
print(f'Предсказанный класс: {predicted_class[0]}')
print(f'Имя предсказанного класса: {class_names[predicted_class[0]]}')

1/1 ————— 0s 109ms/step
Предсказанный класс: 0
Имя предсказанного класса: daisy

```

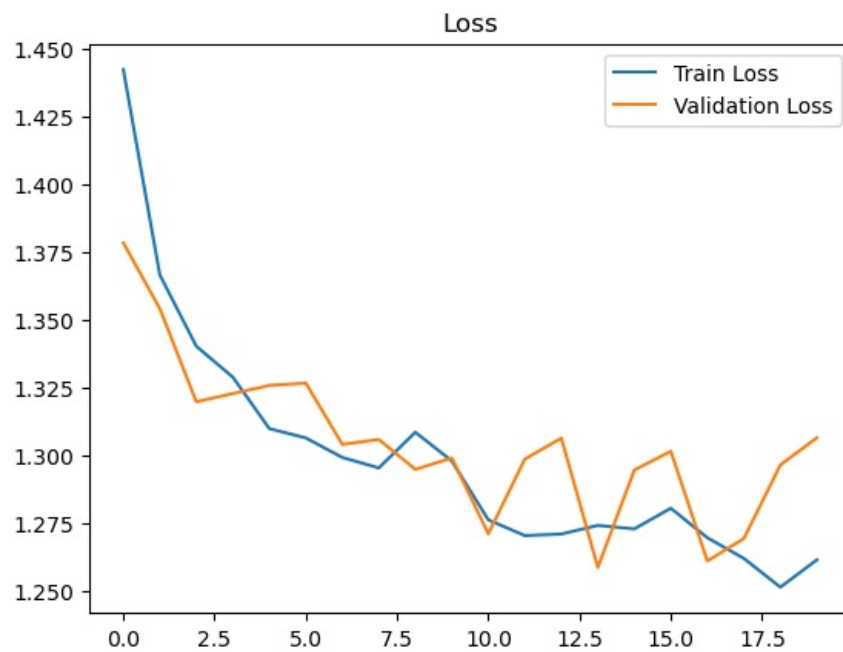
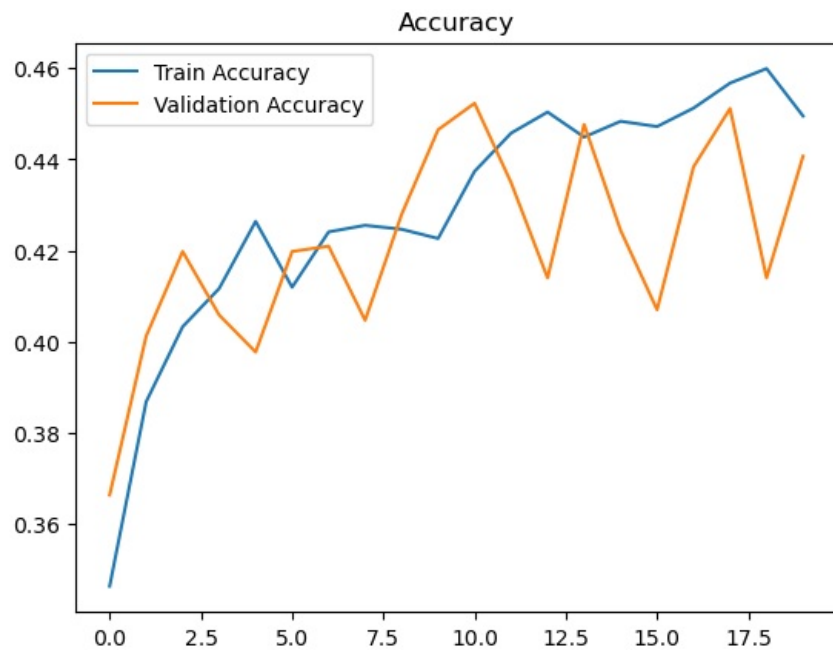
```

In [25]: import matplotlib.pyplot as plt
# График точности
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

```

```
plt.legend()
plt.title('Accuracy')
plt.show()

# График потерь
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.show()
```



In []: